

---

# 嵌入式系统实验指导书

## 试验 4

2025 年 9 月

---

---

# 目录

|                                           |    |
|-------------------------------------------|----|
| 目录.....                                   | 2  |
| 1. 基于 CORTEX-M3 处理器的 KEIL RTX5 移植实验 ..... | 4  |
| 1.1 KEIL RTX5 简介 .....                    | 4  |
| 1.1.1 KEIL RTX5 的特点.....                  | 5  |
| 1.1.2 KEIL RTX5 核心机制 .....                | 6  |
| 1.1.3 KEIL RTX5 源码结构 .....                | 8  |
| 1.2 实验一 KEIL RTX5 系统移植与应用实验 .....         | 11 |
| 1.2.1 实验目的 .....                          | 11 |
| 1.2.2 实验环境.....                           | 11 |
| 1.2.3 实验原理.....                           | 11 |
| 1.2.4 实验内容.....                           | 12 |
| 1.2.5 实验步骤.....                           | 15 |
| 1.2.6 实验现象.....                           | 20 |

---

# 1. 基于 Cortex-M3 处理器的 Keil RTX5 移植实验

## 1.1 Keil RTX5 简介

Keil RTX5 是由 ARM 公司开发的一款开源、确定性的实时操作系统（RTOS），专为基于 ARM Cortex-M 系列处理器的应用程序而设计。作为 ARM Keil MDK (Microcontroller Development Kit) 开发工具链的核心组件之一，RTX5 提供了强大的多任务处理能力，适用于各种对响应速度和系统可靠性有较高要求的嵌入式系统。

RTX5 的一个核心设计理念是完全遵循 CMSIS-RTOS v2 (Cortex Microcontroller Software Interface Standard) API 规范。CMSIS-RTOS 是一套标准化的 API 接口，旨在为嵌入式软件提供统一的、可移植的 RTOS 层。作为该标准的官方参考实现，RTX5 确保了代码的高度可移植性，开发者编写的应用程序可以轻松地在其他同样支持 CMSIS-RTOS v2 接口的操作系统上运行。

它与 Keil MDK 开发环境无缝集成，通过 MDK 的运行时环境（RTE）配置向导，开发者可以非常方便地将 RTX5 添加到项目中，并以图形化的方式配置内核参数、线程数量、堆栈大小以及其他内核对象。这种紧密的集成极大地简化了 RTOS 的使用门槛，并提供了强大的系统级调试支持。RTX5 基于 Apache 2.0 许可协议，开源且免版税，可用于商业产品开发。

### 1.1.1 Keil RTX5 的特点

#### 1. 确定性的多任务内核 (Deterministic Multi-tasking Kernel)

RTX5 采用基于优先级的抢占式调度策略。每个任务（在 RTX5 中称为线程）都有一个独立的优先级，调度器确保 CPU 始终运行在当前处于就绪态的最高优先级任务上。这种机制保证了关键任务的实时响应能力。对于相同优先级的任务，RTX5 还支持时间片轮询调度，确保它们能公平地分享 CPU 时间。每个线程都拥有自己独立的堆栈空间，保证了任务上下文的隔离与安全。

#### 2. 全面遵循 CMSIS-RTOS v2 标准 (Full Compliance with CMSIS-RTOS v2)

RTX5 是 CMSIS-RTOS v2 API 的原生实现。这意味着它的所有功能和服务都通过标准化的接口提供。这不仅使得应用程序逻辑与底层 RTOS 内核解耦，增强了代码的可维护性和可移植性，也让开发者能够利用丰富的 CMSIS 生态系统资源。

#### 3. 丰富的内核对象与服务 (Rich Set of Kernel Objects and Services)

RTX5 提供了实时系统开发所需的全套内核对象，用于任务管理、同步和通信：

- **线程管理 (Thread Management):** 创建、终止和控制任务的执行。
- **同步机制 (Synchronization):** 包含信号量 (Semaphores)、互斥锁 (Mutexes) 和事件标志 (Event Flags)，用于处理资源共享、任务同步等复杂场景。
- **线程间通信 (Inter-Thread Communication):** 提供消息队列 (Message Queues) 和线程标志 (Thread Flags)，允许任务之间安全、高效地传递数据和信令。
- **内存管理 (Memory Management):** 内置了内存池 (Memory Pool) 功能，支持高效的、固定大小的动态内存分配，有效避免内存碎片。
- **软件定时器 (Kernel Timers):** 提供一次性或周期性的软件定时器，用于执行延时操作或周期性任务，而无需占用硬件定时器资源。

#### 4. 与 Keil MDK 无缝集成 (Seamless Integration with Keil MDK)

这是 RTX5 最显著的优势之一。通过 Keil  $\mu$ Vision IDE，开发者可以：

---

- 。使用 **RTE (Runtime Environment)** 图形化配置向导轻松完成内核的配置。
- 。在调试期间，使用 **Component Viewer** 实时查看所有线程的状态、堆栈使用情况、消息队列内容以及其他内核对象的信息。
- 。利用 **System Analyzer** 记录和可视化任务切换、中断服务以及 API 调用，深入分析系统的实时行为。

## 5. 专为资源受限系统优化 (Optimized for Resource-Constrained Systems)

RTX5 的内核设计非常紧凑，占用的代码空间（ROM）和数据空间（RAM）极小。其所有内核对象都是静态配置的，只有在项目中实际使用的功能才会被编译链接，从而最大限度地节省了系统资源，非常适合存储空间和内存有限的微控制器。

## 6. 支持功能安全 (Functional Safety Support)

RTX5 的设计考虑了功能安全标准（如 IEC 61508 和 ISO 26262），并提供了相关的安全认证包（需额外授权），使其能够被应用于汽车、医疗和工业控制等对可靠性要求极高的安全关键领域。

### 1.1.2 Keil RTX5 核心机制

RTX5 的核心主要由以下几个关键机制构成：

#### 1. 优先级抢占式调度 (Priority-Based Preemptive Scheduling)

这是 RTX5 最核心的调度机制。系统中的每一个线程（Thread）都被分配了一个优先级。调度器（Scheduler）是内核的心脏，它的唯一工作就是确保 **CPU** 始终运行在当前所有“就绪态”（Ready）线程中优先级最高的那一个。

- **抢占 (Preemption):** 当一个正在运行的低优先级线程（如 Thread\_A）执行时，如果一个更高优先级的线程（Thread\_B）因为某个事件（例如：中断服务程序释放了一个信号量、一个延时结束）而变为就绪态，调度器会立即中断 Thread\_A 的执行，转而去执行 Thread\_B。只有当 Thread\_B 执行完毕或因等待资源而进入阻塞态（Blocked）时，Thread\_A 才有机会重新获得 CPU 的使用权。这保证了关键任务总能被优先处理。

- **时间片轮询 (Round-Robin):** 对于多个具有相同优先级的就绪态线程，调度器会采用时间片轮询的方式来调度它们。每个线程会被分配一个固定的时

间片（Ticks），当一个线程的时间片用完后，如果它没有阻塞，调度器会自动切换到同优先级的下一个就绪线程。这确保了同级任务能够公平地共享 CPU 资源。

## 2. 独立的线程堆栈与上下文切换 (Independent Thread Stacks & Context Switching)

这个机制与 Contiki 的 Protothread 共享堆栈模型截然相反，是 RTX5 实现安全多任务的基础。

- **独立堆栈 (Independent Stacks):** 在 RTX5 中，**每一个线程都拥有自己独立的内存堆栈空间**。这个堆栈用于存储线程的局部变量、函数调用返回地址以及最重要的——**线程的上下文（Context）**。

- **上下文切换 (Context Switching):** 上下文指的是一个线程在被中断时刻的所有 CPU 寄存器的状态（包括通用寄存器 R0-R12、程序计数器 PC、程序状态寄存器 PSR 等）。当调度器决定要进行线程切换时，会执行以下原子操作：

- 1) **保存当前线程上下文:** 将当前 CPU 的所有寄存器值压入该线程自身的堆栈中进行保存。
- 2) **恢复下一个线程上下文:** 从即将要运行的高优先级线程的堆栈中，将它上一次被保存的寄存器值弹出，恢复到 CPU 的各个寄存器中。

这个“保存-恢复”的过程就是上下文切换。完成后，CPU 的程序计数器（PC）就指向了新线程上一次被中断的代码位置，新线程得以继续执行，仿佛它从未被打断过。虽然上下文切换会带来一定的开销，但它彻底隔离了各个线程的运行环境，极大地提高了系统的稳定性和可靠性。

## 3. 内核节拍与系统定时器 (Kernel Tick & System Timer)

为了实现线程延时、超时等待以及时间片轮询等与时间相关的功能，RTX5 需要一个周期性的“心跳”，这个心跳被称为**内核节拍（Kernel Tick）**。

在 Cortex-M 处理器上，RTX5 通常使用内核自带的 **SysTick 定时器** 来实现这个机制。SysTick 被配置为以固定的频率（例如 1ms 或 10ms）产生一个周期性中断。在 SysTick 中断服务程序（ISR）中，内核会执行以下操作：

- 更新系统时钟计数。
- 检查所有处于阻塞态的线程，看它们的延时或等待是否超时。如果超时，则将该线程重新置为就绪态。

- 
- 处理软件定时器的超时事件。
  - 执行时间片轮询调度。
  - 在处理完上述逻辑后，如果发现有更高优先级的线程变为就绪态，则触发一次上下文切换。

#### 4. 内核服务调用 (Kernel Service Calls)

为了保证内核数据结构的安全，线程不能直接修改内核对象。当线程需要请求操作系统服务时（例如申请一个互斥锁 `osMutexAcquire` 或延时 `osDelay`），它必须通过特定的 CPU 指令发起 **SVC（Supervisor Call，服务调用）**。该指令会触发一个 SVC 异常，使 CPU 进入特权级的异常处理模式来安全地执行内核代码。执行完内核服务后，再返回到线程代码。这个机制保证了操作系统的稳定运行，防止用户代码破坏内核。

### 1.1.3 Keil RTX5 源码结构

与 Contiki-OS 需要手动拷贝和组织源文件文件夹的模式不同，Keil RTX5 的源码结构是通过\*\*Keil MDK 的运行环境（RTE, Runtime Environment）\*\*进行管理的。开发者通常不直接接触 RTX5 的全部源文件，而是通过图形化界面来选择和配置所需的软件组件。

这种组件化的结构旨在简化项目管理、增强代码的可移植性并确保组件间的兼容性。在 Keil MDK 的 Project 窗口中，一个集成了 RTX5 的项目的源码结构主要由以下几个逻辑部分组成：

#### 1. CMSIS-CORE 组件

这是所有基于 Cortex-M 处理器的项目的基础。它提供了访问核心处理器寄存器（如 NVIC、SysTick）的标准 API。在移植 RTOS 时，这部分至关重要，因为 RTOS 内核需要通过它来配置系统心跳定时器（SysTick）和管理中断。

关键文件：

`core_cmX.h`: 针对具体 Cortex-M 内核（如 M3, M4）的头文件。

`system_stm32f10x.c`: 芯片级的系统配置文件，通常用于初始化系统时钟。

`startup_stm32f10x.s`: 汇编启动文件，包含了中断向量表。RTX5 会通过它来接管 SysTick\_Handler, PendSV\_Handler, 和 SVC\_Handler 等关键系统异常。

#### 2. RTOS2 API 组件 (CMSIS::RTOS2)



---

这是应用程序员直接打交道的接口层（API）。它定义了一套标准化的函数，如 `osThreadNew`, `osMutexAcquire`, `osDelay` 等。开发者在编写应用代码时，包含的是这个 API 的头文件，而不是 RTOS 内核的具体实现文件。

关键文件：

`cmsis_os2.h`: 定义了所有 CMSIS-RTOS v2 标准接口的头文件。这是你在应用程序中需要 `#include` 的核心文件。

### 3. RTOS 内核实现组件 (CMSIS::RTOS:Keil RTX5)

这是 RTOS 的核心实现层。它包含了实现 CMSIS-RTOS2 API 背后所有功能的源代码，包括调度器、线程管理、内存管理、同步与通信机制等。这些文件由 Keil MDK 根据你在 RTE 中的选择自动添加到项目中。通常情况下，你不需要也不应该去修改这些内核源文件。

核心源文件（示例）：

`rtx_kernel.c`: 内核的核心，包含系统初始化和调度器逻辑。

`rtx_thread.c`: 实现了线程创建、管理和上下文切换的功能。

`rtx_mutex.c`: 互斥锁的实现代码。

`rtx_semaphore.c`: 信号量的实现代码。

`rtx_delay.c`: 延时功能的实现代码。

... 其他内核对象的实现文件。

### 4. RTOS 配置组件

这是开发者配置 RTOS 内核行为的地方。Keil MDK 提供了一个图形化的配置向导，让你能够方便地设置系统参数。这些设置最终会保存在一个配置文件中。

关键文件：

`RTX_Config.h`: 这是 RTX5 的核心配置文件。它定义了诸如系统定时器频率、默认线程堆栈大小、内核对象的数量、是否启用时间片轮询等重要参数。你可以通过 Keil MDK 提供的图形化配置工具（打开该文件时自动弹出）来修改它，也可以直接编辑这个头文件。

总结来说，Keil RTX5 的源码结构在用户的视角下是逻辑化的、组件化的：

应用层：你编写的代码，包含 `cmsis_os2.h` 并调用 `os...` 系列函数。

配置层：你通过图形化界面修改 `RTX_Config.h` 来调整 RTOS 的行为。

接口层 (API)：由 `cmsis_os2.h` 定义，是应用层和内核层之间的桥梁。

内核实现层：由 RTE 自动管理的 `rtx_*.c` 文件，你无需关心其内部细节。

硬件抽象层：由 CMSIS-CORE 和启动文件提供，将内核与具体硬件连接起来。

这种结构极大地简化了 RTOS 的集成和移植工作，使开发者能更专注于应用程序的开发。

---

## 1.2 实验一 Keil RTX5 系统移植与应用实验

### 1.2.1 实验目的

1. 理解 Keil RTX5 抢占式实时操作系统的核心工作原理。
2. 掌握在 Keil MDK 5 开发环境中，通过运行时环境（RTE）为 STM32 项目添加并配置 RTX5 内核的方法。
3. 学会创建 RTOS 线程，并使用 RTOS 的 API（如延时、线程间通信）编写一个简单的多任务应用程序。

### 1.2.2 实验环境

1. 硬件：硬件：STM32 无线节点板一个、下载调试板、ARM JLink 仿真器、USB 方口线、PC 机、MicroUSB 线/交叉串口线、DC5V 电源；
2. 软件：Windows7 及以上系统，Keil5 MDK-ARM5.43a，串口调试助手 SuperCom。

### 1.2.3 实验原理

本次实验的核心是利用 Keil MDK 的运行时环境（RTE）将 RTX5 操作系统组件化地集成到 STM32 项目中。与传统的需要手动拷贝大量源码、修改头文件和编写启动代码的 RTOS 移植方式（如移植 uCOS-II）相比，基于 RTE 的移植过程实现了高度的自动化和图形化。

其基本原理如下：

1. 组件化管理：Keil MDK 通过 Pack Installer 下载并管理包含了设备驱动、中间件（如 RTOS、文件系统）的软件包（Pack）。RTX5 本身就是一个标准的软件包（ARM::CMSIS）。
2. 自动集成：在创建项目后，开发者通过 RTE 图形化界面选择所需的软件组件（例如：CMSIS-CORE、Device Startup、RTOS Kernel）。Keil MDK 会自动将这些组件的源文件和头文件包含到项目中，并处理好它们之间的依赖关系。
3. 图形化配置：RTX5 内核的各项参数，如系统时钟频率、默认堆栈大小、是否启用时间片轮询等，都可以在一个名为 RTX\_Config.h 的配置文件中进

---

设置。Keil MDK 为这个文件提供了友好的图形化配置界面，大大降低了配置的复杂度和出错率。

4. 标准 API 调用：RTX5 完全遵循 CMSIS-RTOS v2 API 标准。应用程序通过包含 `cmsis_os2.h` 头文件，调用 `osThreadNew`、`osDelay` 等标准函数来使用 RTOS 功能。RTOS 内核的关键异常（如 `SysTick_Handler`、`PendSV_Handler`、`SVC_Handler`）由 RTX5 组件自动实现并接管，开发者无需手动编写。

## 1.2.4 实验内容

本次实验的核心内容是在 Keil MDK 5 开发环境中，为一个基于 STM32F103 微控制器的项目移植并应用 Keil RTX5 实时操作系统。

我们将通过以下具体任务来完成本次实验：

1. 创建并配置 RTOS 工程：学习使用 Keil MDK 的运行时环境（RTE）来新建一个工程，并自动地将 RTX5 内核及相关的 CMSIS 组件集成到项目中。
  2. 实现硬件驱动：编写基本的硬件外设初始化代码，包括用于控制 LED 灯的 GPIO 端口和用于数据输出的 USART 串口。
  3. 创建两个并发线程：
    - LED 闪烁线程 (`Thread_LED`)：该线程的任务是周期性地改变一个 LED 灯的状态，使其以固定的频率（例如：每 500 毫秒）闪烁。
    - 串口打印线程 (`Thread_UART`)：该线程的任务是周期性地通过 USART 串口向 PC 机发送一条状态信息（例如：“RTOS is running...”），其频率与 LED 闪烁线程不同（例如：每 1000 毫秒）。
  4. 验证多任务调度：通过观察 LED 的闪烁和串口接收到的信息，验证 RTX5 调度器能够正确地在两个线程之间进行切换，使它们看起来像是“同时”在运行，从而直观地理解 RTOS 的并发性。
    - 1) 打开 Manage Run-Time Environment 窗口中，按以下步骤勾选所需组件：  
CMSIS: 展开并勾选 CORE。  
CMSIS: 展开 RTOS2 (API)，选择 Keil RTX5。  
Device: 展开并勾选 Startup。  
Device: 展开 StdPeriph Drivers，勾选 Framework, GPIO, RCC 和 USART。
-

如图 1-1 所示。

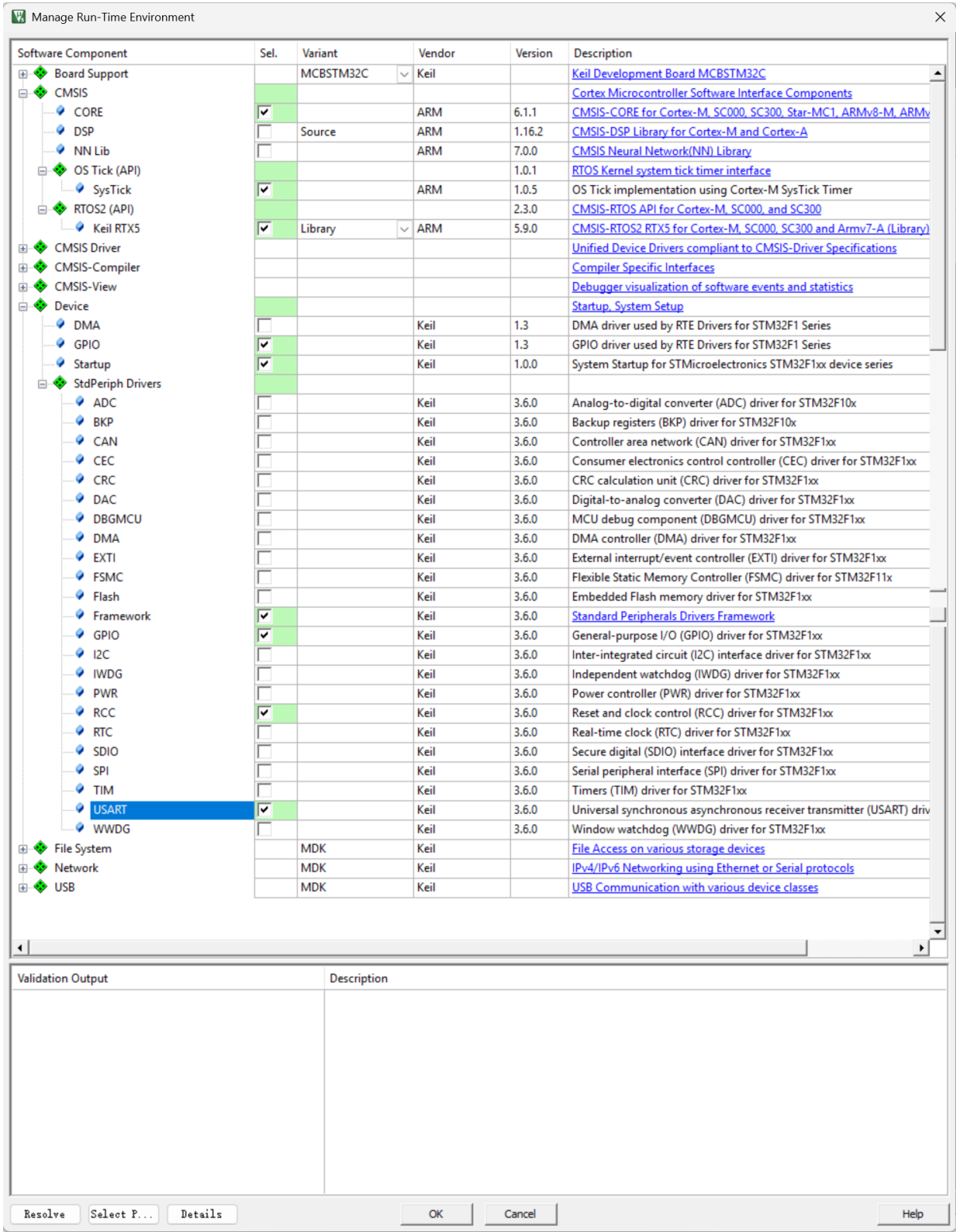


图 1-1 Manage Run-Time Environment 窗口

2) 点击 Resolve 按钮，让 Keil 自动处理组件间的依赖关系，然后点击 OK。

- 3) 点开 Project 栏的 RTX\_Config.h, 把 OS\_DYNAMIC\_MEM\_SIZE 的定义值改为 8192

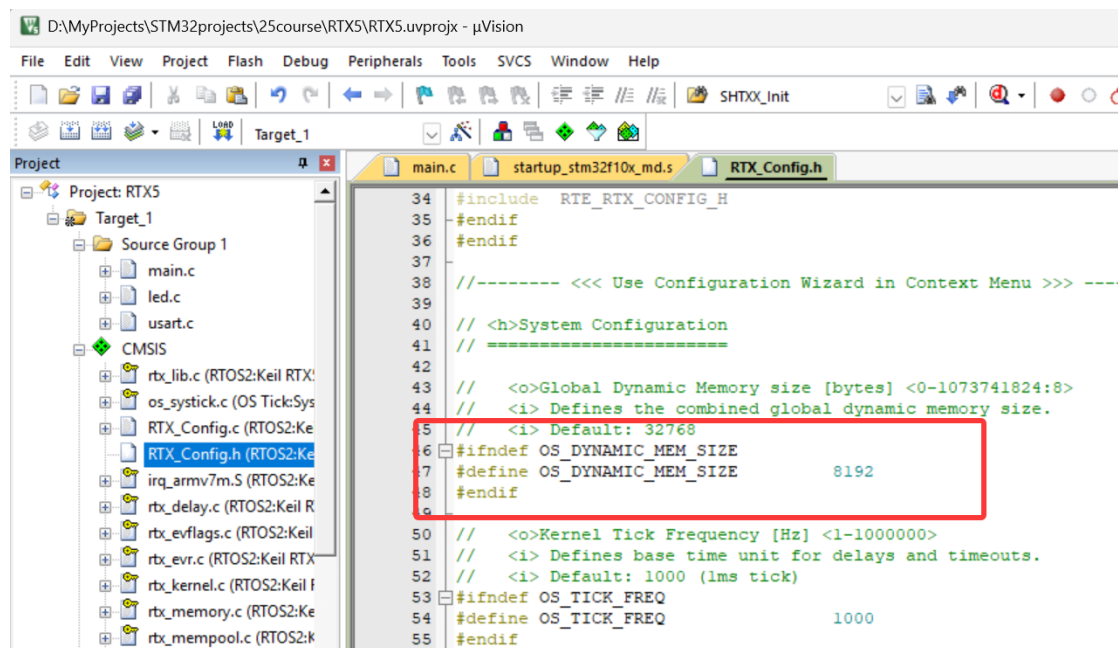


图 1-2 修改 RTX\_Config.h

- 4) 复用之前实验的 led.c, led.h, usart.c, usart.h, 把他们添加进项目

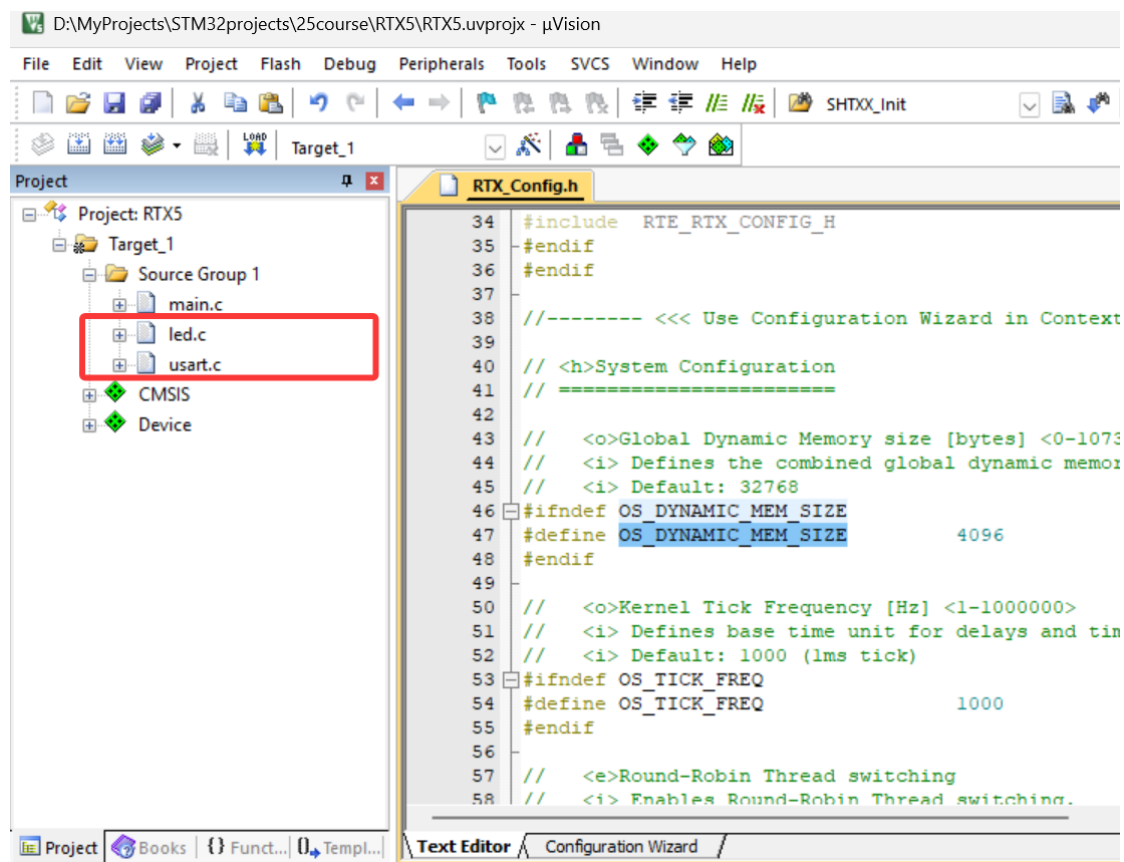


图 1-3 添加源代码

## 5) 新建 main.c

```
#include "RTE_Components.h"
#include "stm32f10x.h"
#include "rtx_os.h"
#include <stdio.h>
#include "led.h"
#include "usart.h"

void Hardware_Init(void) {
    SystemInit();
    leds_init();
    uart1_init();
}

// LED 闪烁线程的实现
void thread_led_task(void *argument) {
    uint8_t led_state = 0;
    for (;;) {
        led_state = !led_state;
        if (led_state) {
            D1_on(); // 点亮 LED (低电平有效)
        } else {
            D1_off(); // 熄灭 LED
        }
        osDelay(500); // 挂起线程 500ms
    }
}

// 串口打印线程的实现
void thread_uart_task(void *argument) {
    uint32_t counter = 0;
    for (;;) {
        printf("Message %lu: RTOS is running...\r\n", ++counter);
        osDelay(1000); // 挂起线程 1000ms
    }
}

// 主函数
int main(void) {
    Hardware_Init();
    printf("Hardware Initialized. Starting RTOS Kernel...\r\n");

    osKernelInitialize();           // 初始化 RTOS 内核
    osThreadNew(thread_led_task, NULL, NULL); // 创建 LED 线程
    osThreadNew(thread_uart_task, NULL, NULL); // 创建 UART 线程
    osKernelStart();               // 启动内核调度

    for (;;) // 程序不应执行到这里
}
```

### 1.2.5 实验步骤

- 1) 使用 USB 方口线连接 ARM J-LINK 仿真器和 PC 机，用 20 针排线连接 AR

M J-LINK 仿真器和下载调试板，10 针排线连接下载调试板和 STM32 开发板，再使用串口线将下载调试板连接至 PC，使用 DC 5V 电源给开发板供电，如图 1-4 所示。

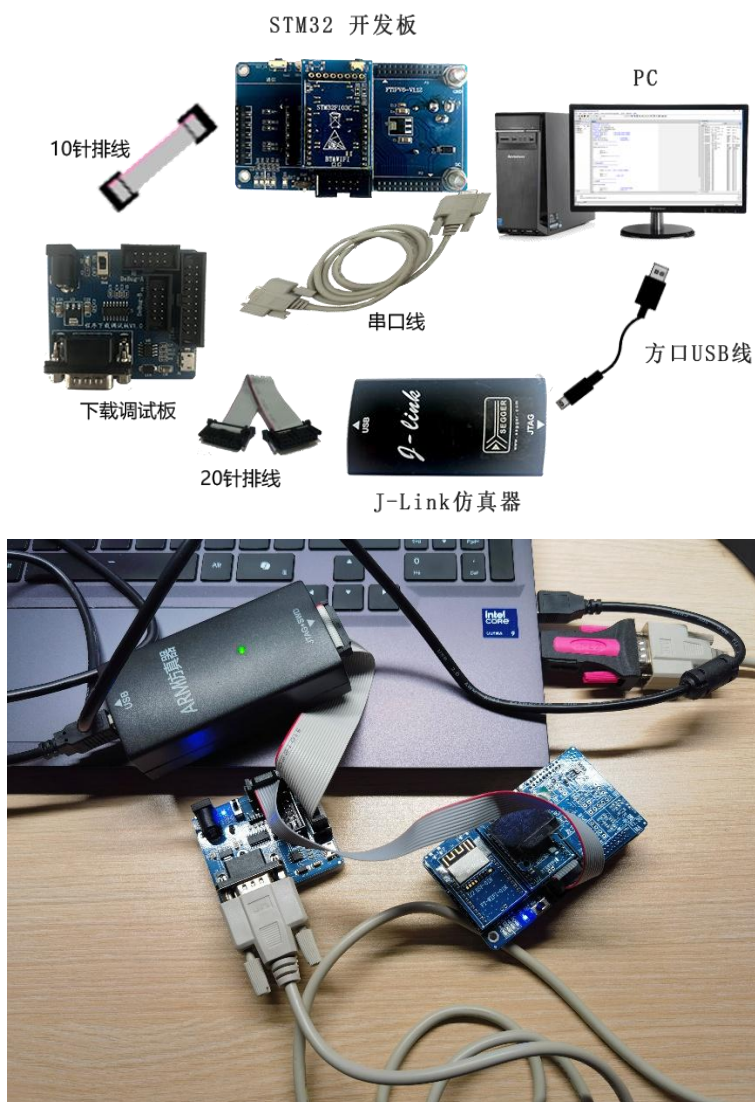


图 1-4 硬件连接图

- 2) 在 PC 机上查看 STM32 开发板连接到 PC 机的端口号，如果串口线连接到 PC 机的 DB9 口，通常端口号默认为 COM1，如果串口线是通过 U 转串连接到电脑 USB 口上的，则需要打开设备管理器查看正确的端口号，如图 1-5 所示，打开设备管理器（不同的 Windows 系统打开方式可能不同）。





图 1-5 打开设备管理器（左 win10，右 win11）

然后在设备管理器的端口下查看串口使用的端口号，如图 1-6 所示。

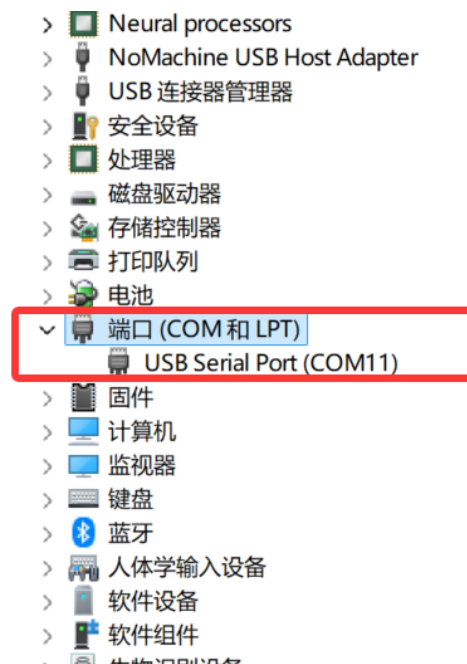


图 1-6 在设备管理器的端口下查看端口号

3) 用 Keil5 开发环境打开实验例程，点击 **Rebuild All** 重新编译工程，如图

1-7 所示。

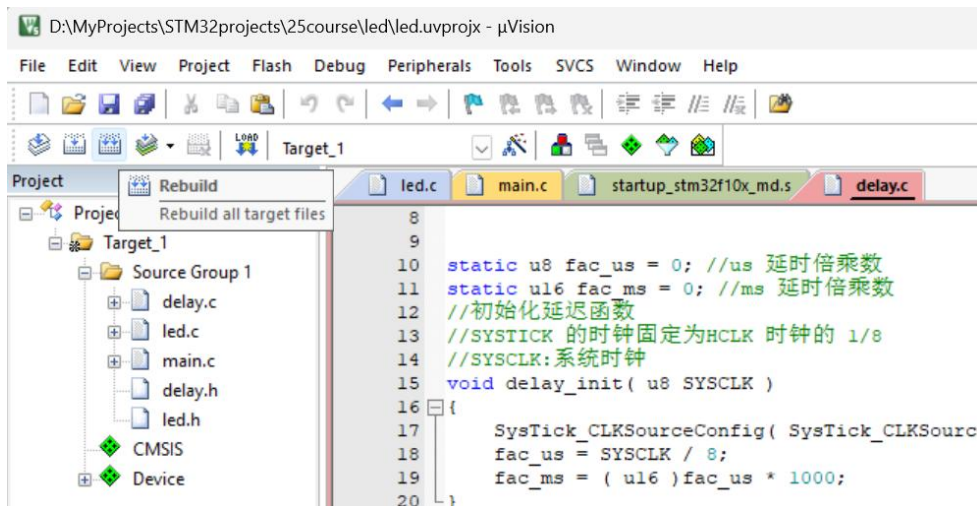


图 1-7 重新编译

- 4) 点击 Download 将程序下载到 STM32 开发板中，即可观察到开发板程序运行（如按照教程勾选”Reset and Run”）如图 1-8 所示。也可以将 STM32 开发板重新上电或者按下复位按钮让刚才下载的程序重新运行。

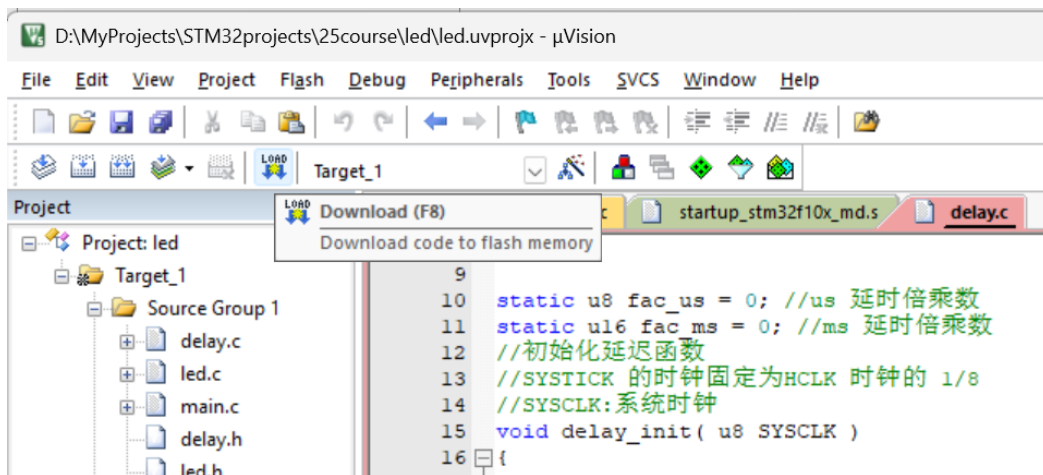


图 1-8 下载程序

- 5) （可选）下载完后可以点击 Debug 让程序进入调试模式，如图 1-9 所示；

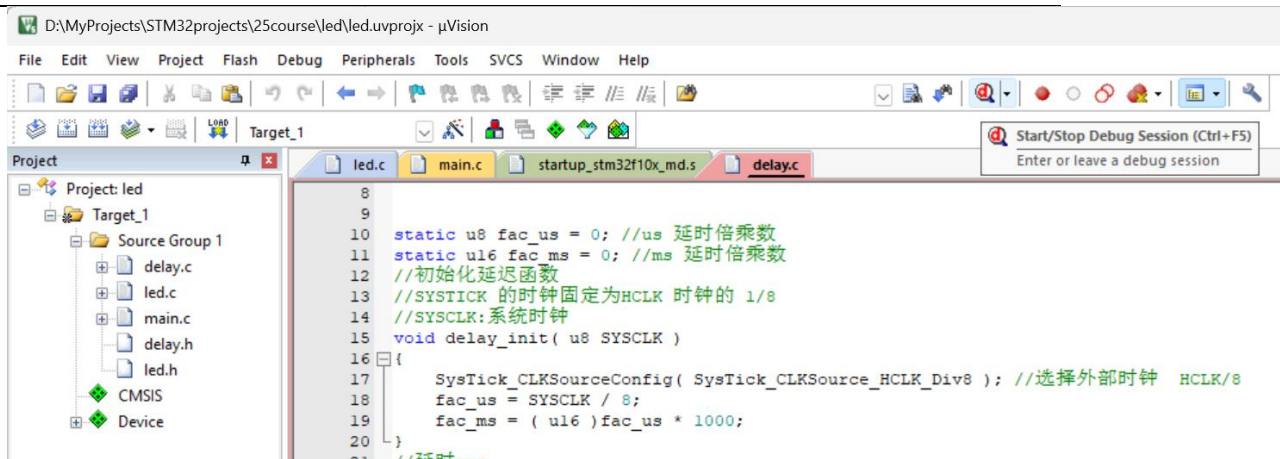


图 1-9 调试程序

- 6) 程序成功运行后，在 PC 机上打开 SuperCom.exe (直接解压压缩包，程序在解压后的文件夹内)，设置波特率为 115200，端口号选择步骤 2 查看到的端口，不勾选自动清空，其他设置采用默认值，点击打开串口，如图 1-10 所示。

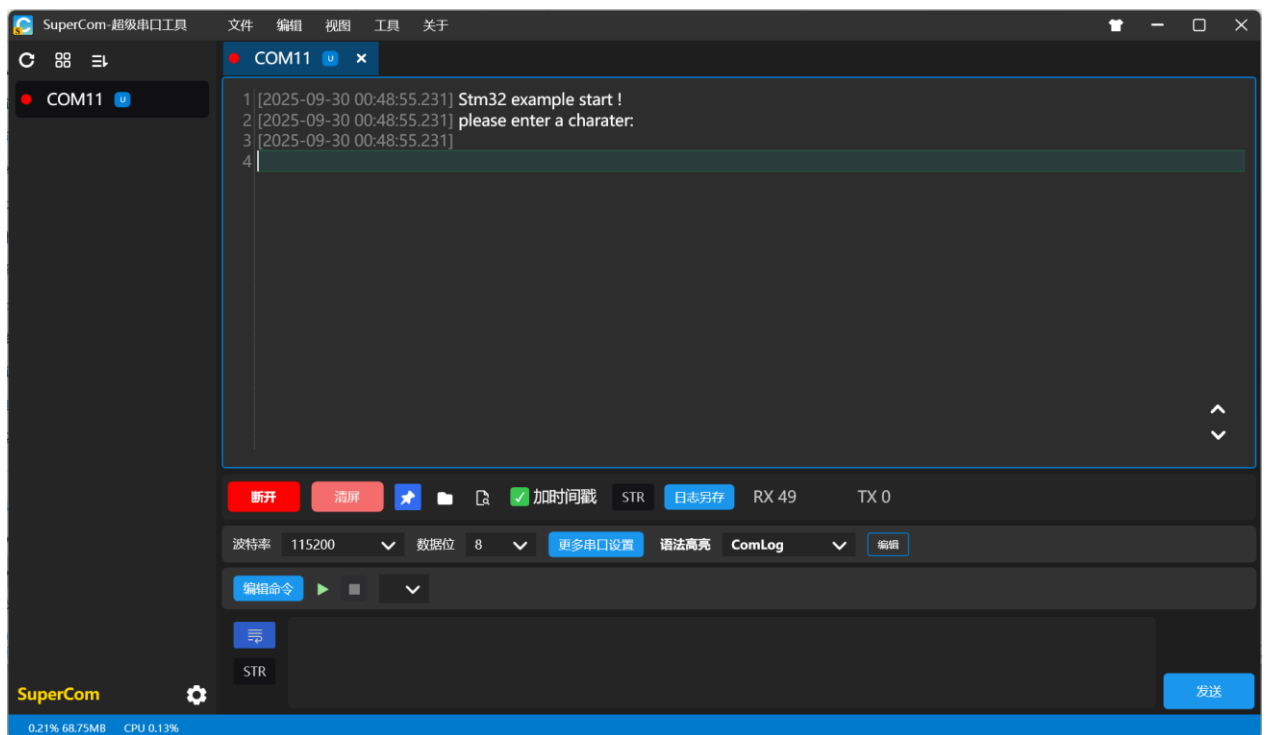


图 1-10 打开串口助手

### 1.2.6 实验现象

程序下载并运行后，你将同时观察到以下两个现象：

1. 视觉现象：开发板上的 LED 灯会以稳定的频率进行闪烁，亮 500 毫秒，灭 500 毫秒，即每秒闪烁一次。
2. 串口现象：打开 PC 上的串口调试助手（波特率设置为 115200），你会看到每隔 1 秒钟，接收窗口就会打印出一条新的信息，并且计数器递增。

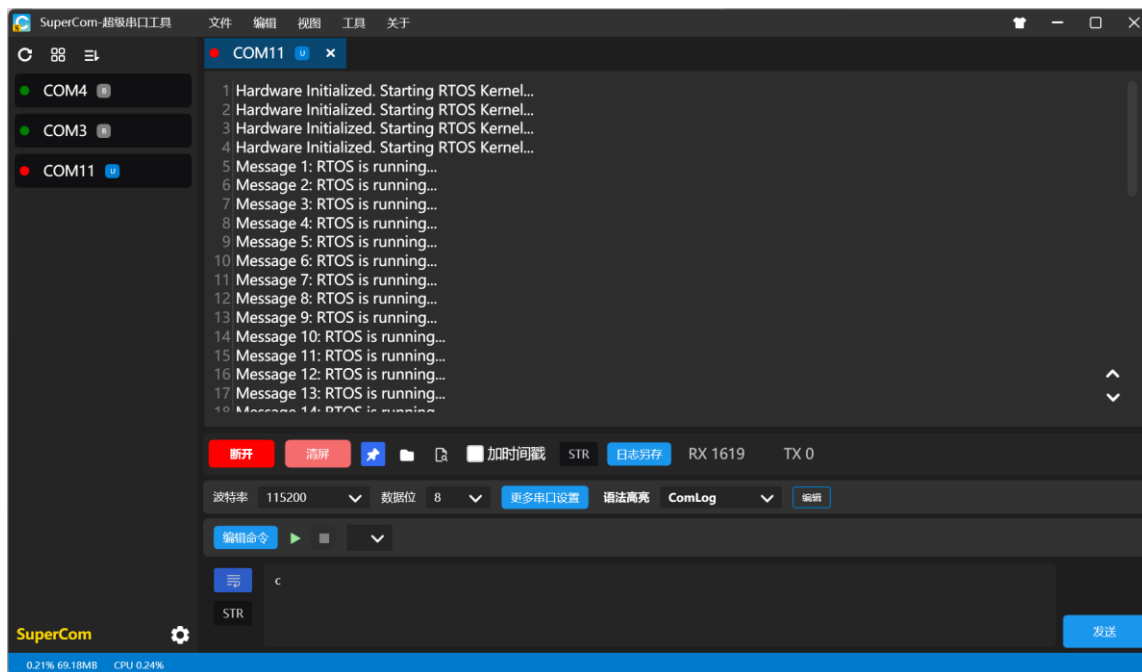


图 1-11 实验现象

这两个任务（LED 闪烁和串口打印）以不同的周期独立、并发地运行，互不干扰。这清晰地展示了 Keil RTX5 抢占式多任务内核的调度效果，证明操作系统已成功移植并正常工作。